

Day 2 — Python Core II

Functions, Comprehensions & File I/O — Notes for Recap

1. Functions & Default Arguments

What is a function?

A function is a reusable block of code that performs a specific task. Instead of repeating the same code, you define it once and call it whenever needed.

```
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet("Akash"))    # Hello, Akash!
```

Default arguments

Default arguments allow a parameter to have a fallback value if no argument is passed when calling the function.

```
def greet(name="Guest"):  
    return f"Hello, {name}!"  
  
print(greet())           # Hello, Guest!  
print(greet("Akash"))    # Hello, Akash!
```

Key takeaway: Functions make code reusable and organized. Default arguments make functions more flexible — useful in data pipelines where a parameter (like a file path or batch size) often has a sensible default but can be overridden.

2. List/Dict Comprehensions

What is a comprehension?

A comprehension is a compact, one-line way to create a list or dictionary by transforming or filtering an existing collection — instead of writing a full for loop.

List comprehension:

```
numbers = [1, 2, 3, 4, 5]  
squares = [n**2 for n in numbers]  
print(squares)    # [1, 4, 9, 16, 25]  
  
# with a condition  
even_squares = [n**2 for n in numbers if n % 2 == 0]  
print(even_squares)    # [4, 16]
```

Dictionary comprehension:

```
names = ["akash", "priya", "raj"]
name_lengths = {name: len(name) for name in names}
print(name_lengths)  # {'akash': 5, 'priya': 5, 'raj': 3}
```

Key takeaway: Comprehensions are shorter and often faster than writing an explicit loop. In data engineering, they're commonly used for quick transformations — e.g., cleaning a list of values or reshaping a row into a dictionary.

3. Modules & Imports

What is a module?

A module is simply a Python file (or built-in library) containing reusable code — functions, classes, or variables — that can be imported into another script.

```
import math
print(math.sqrt(16))  # 4.0

from datetime import datetime
print(datetime.now())

import pandas as pd  # common alias in data work
```

Types of imports

Syntax	What it does
import module	Imports the whole module
from module import x	Imports only a specific function/class
import module as alias	Imports with a shorter nickname

Key takeaway: Modules let you avoid reinventing the wheel — data engineering relies heavily on external modules like pandas, os, json, and requests rather than writing everything from scratch.

4. Exception Handling (try / except)

What is exception handling?

It's a way to handle errors gracefully instead of letting the program crash when something unexpected happens.

```
try:
    value = int("abc")  # this will fail
except ValueError:
    print("That wasn't a valid number.")
```

Common structure:

```
try:
    # risky code
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero.")
except Exception as e:
    print(f"Something went wrong: {e}")
finally:
    print("This always runs, error or not.")
```

- try — code that might fail
- except — runs only if an error occurs
- finally — runs no matter what (often used for cleanup, like closing a file)

Key takeaway: *Real-world data is messy — missing files, bad values, failed API calls. Exception handling prevents one bad record from crashing an entire pipeline, and lets you log the issue instead.*

5. Reading/Writing Files

Why this matters: Data engineering constantly involves reading data in and writing results out — CSVs, logs, config files, etc.

Reading a file:

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

Writing to a file:

```
with open("output.txt", "w") as f:
    f.write("Pipeline completed successfully.\n")
```

Why use with?

The with statement automatically closes the file when done — even if an error occurs. Without it, you'd have to manually call `f.close()`, and forgetting to do so can lead to file corruption or memory issues.

Mode	Meaning
"r"	Read (file must exist)
"w"	Write (overwrites existing content)
"a"	Append (adds to the end)

Key takeaway: *Always use `with open(...)` for file handling — it's the safe, standard pattern used throughout real data pipelines.*

Day 2 Summary — Python Core II

Concept	One-Line Recap
Functions & Default Args	Reusable blocks of code; default values make them flexible
List/Dict Comprehensions	Compact syntax for building lists/dicts from existing data
Modules & Imports	Reusing external code instead of rewriting it
Exception Handling	Catching errors gracefully so one bad record doesn't crash the pipeline
Reading/Writing Files	Standard, safe way to read from and write to files using with open(...)

Overall takeaway: Day 2 builds directly on Day 1 — functions and comprehensions make your code cleaner and more reusable, while exception handling and file I/O make it robust enough to survive real-world, messy data.